

CFRNet: Cycle-Consistent Fixed-Point Training for Real-Time Blind Face Restoration on Consumer Embedded NPUs

Fuchen Li, Xinyang Wang, Yahui Zhang, Yuhan Chen, Jiahong Guo, Zhuohan Qin, and Wenbo Ma

Abstract—Blind face restoration on consumer devices has to balance image quality against speed and memory. Strong methods such as GFPGAN and CodeFormer give good perceptual quality, but they rely on large pretrained generative priors and on operators such as attention, codebook lookup, and style modulation that are hard to compile and quantize on the small neural processing units (NPUs) used in consumer hardware. Small convolutional restorers run fast enough, but they tend to over-smooth and to leave artifacts around the eyes, nose, and mouth. We present CFRNet, a 2.0 M-parameter ResNet-style restorer for on-device use at 256×256 , the common face-crop size on consumer NPUs. The main idea is Cycle-Consistent Fixed-Point Training (CCFP). Instead of training the network for one pass and then running it several times by hand, we train it to act as a fixed-point operator, so that applying it again to a restored face does not change the face. CCFP uses three training losses, namely progressive multi-cycle supervision, an idempotence loss, and a re-degradation cycle loss, and it adds no cost at inference. To compare fairly under our deployment limits, we retrain all baselines from scratch at the same 256×256 resolution. On a 300-image test set, CFRNet reaches the best perceptual score (LPIPS 0.250 at three cycles, which is 31% lower than one cycle) and also the best PSNR and SSIM at two cycles. It runs in about 23 ms per cycle in INT8 on a HiSilicon Hi3402 NPU, while the same baselines cannot be compiled to that chip. The cycle count k acts as a simple quality knob that needs no retraining: PSNR is best at $k = 2$ and LPIPS keeps improving up to $k = 3$. We further show that the same idea works with a plain CNN that is even easier to deploy, and we run the model in real time on an in-car driver-monitoring board.

Index Terms—Blind face restoration, cyclical inference, fixed-point training, cycle consistency, lightweight neural networks, embedded NPU, consumer electronics, on-device inference.

I. INTRODUCTION

BLIND face restoration takes a degraded face image, which may have blur, downsampling, noise, and compression artifacts, and recovers a clean and identity-consistent face. The task matters for many consumer products: photo and video apps on phones, video calls, smart cameras and doorbells, and in-car driver-monitoring systems [1], [2]. In these products the input is often captured in poor conditions,

and the model has to run on a small NPU with a tight latency and memory budget. Recent prior-guided methods such as GFPGAN [3], GPEN [4], RestoreFormer [5], and CodeFormer [6] give very good perceptual quality. They do this with pretrained StyleGAN2 generators, transformer modules, or learned codebooks. The cost is size and complexity. Counting the frozen priors, these systems often have 70–85 M parameters, and they use operators such as style modulation, multi-head attention, and codebook lookup. These operators are hard to compile and to quantize to INT8 on the NPUs found in consumer devices. Small CNN restorers [7], [8] fit the latency budget, but they have two common problems: they over-smooth high-frequency texture, and they leave artifacts around the eyes, nose, and mouth. One way to make a small network better without making it larger is to run it more than once at test time. Earlier work on iterative super-resolution [9], [10] and recurrent restoration [11] ran a single network several times. But these methods train the network for a single forward pass and then reuse it in a heuristic way. This leaves a basic question open: why should running a single-pass network again make the result better instead of worse? What stops the second pass from harming a face that is already restored? If G is trained only to map a degraded image x to a clean image y , then feeding it $y' \approx y$ is out of distribution. The network was never asked to keep a good image good, so further passes may over-sharpen, add detail that is not there, or oscillate. We call this the training-inference gap.

This paper closes that gap. We present CFRNet, a small restorer for consumer NPUs, and Cycle-Consistent Fixed-Point Training (CCFP). Rather than train G for one mapping and reuse it by hand, CCFP trains G to be a fixed-point operator on the set of natural faces. Three losses work together:

- **Progressive supervision.** We supervise the output of every cycle $G^{(i)}(x)$, $i = 1, 2, 3$, against the ground truth, with larger weight on later cycles. This teaches G a coarse-to-fine refinement instead of a one-shot mapping.
- **Idempotence.** A loss $\|G(G^{(k)}(x)) - G^{(k)}(x)\|_1$ pushes the last output toward a fixed point of G , so one more pass does not change it.
- **Re-degradation cycle.** If we degrade a restored face again with the degradation model D and restore it, we should get the same restored face: $G(D(G^{(k)}(x))) \approx G^{(k)}(x)$. This pushes G to learn the inverse of the degradation process, not to memorize training pairs.

CCFP only changes training. The deployed network is the

F. Li and X. Wang contributed equally to this work. (Corresponding author: Fuchen Li, (e-mail: fuchen.li@ufl.edu).)

F. Li, X. Wang, and J. Guo are with the University of Florida, Gainesville, FL, USA.

Y. Zhang is with the University of Southampton, Southampton, UK.

Y. Chen is with Chongqing University, Chongqing, China.

Z. Qin is with Qingdao University, Qingdao, China.

W. Ma is with Intel Asia-Pacific Research & Development Ltd, Shanghai, China.

same small model, run $k = 3$ times. The difference is that its iterative behavior is now designed, not accidental. Our target chip is the HiSilicon Hi3402 NPU, a part used in consumer and automotive vision products. It compiles INT8 graphs of standard CNN operators and runs at a 256×256 face crop. The official GFPGAN, GPEN, and CodeFormer releases run at 512×512 and use operators that the Hi3402 toolchain does not support or that lose too much quality in INT8. To make a fair comparison under these limits, we reimplement each baseline as a deploy-compatible “Lite” model with no frozen prior and no transformer or codebook modules, and we train all of them from scratch on the same data, the same degradation, the same 256×256 resolution, and the same schedule as CFRNet. We do not claim that CFRNet beats the full official models at 512×512 . We show that, under limits that a consumer NPU can actually meet, CFRNet gives the best perceptual quality among comparable from-scratch CNN baselines. We also observe that, on the same inputs, CFRNet at $k=3$ reaches about the same perceptual quality as the full official GFPGAN, while using a small fraction of its parameters and compute (Section IV-B). Our contributions are as follows.

- 1) CCFP, a training method that links cyclical inference to a fixed-point objective on the natural-face set. It combines progressive multi-cycle supervision, idempotence, and a re-degradation cycle in one framework.
- 2) A small ResNet-style generator with 2.0M parameters that uses only standard CNN operators, runs in INT8, and takes about 23 ms per cycle on the Hi3402 NPU.
- 3) A nose region added to the usual GFPGAN-style component supervision, which removes a common mid-face artifact of small CNN restorers.
- 4) A perception-distortion trade-off across cycles: PSNR is best at $k = 2$, while LPIPS keeps improving up to $k = 3$. The cycle count is a quality knob that needs no retraining.
- 5) Evidence that the same iterative idea transfers to a plain, easy-to-deploy CNN that reaches $k=3$ quality on hard real inputs, plus a real-time in-car deployment on the embedded board.

II. RELATED WORK

A. GAN-based and prior-guided face restoration

Early CNN restorers improved fidelity [12], [13], [14], [15] but gave over-smooth outputs under strong degradation. SRGAN and ESRGAN showed that adversarial training helps perceptual sharpness [16], [17], [18], and face-specific methods used geometric or identity priors [19], [20]. Prior-guided methods use strong generators: GFPGAN injects features from a pretrained StyleGAN2 [3], GPEN embeds a GAN prior in a U-shaped decoder [4], and RestoreFormer [5] and CodeFormer [6] use transformers and discrete codebooks [21]. These give strong perceptual quality, but the full systems are large and their main modules do not map well to consumer NPU compilation or INT8.

B. Component-aware supervision

Local supervision on face parts appears in landmark- and parsing-guided methods [20] and in component-dictionary

methods [22]. GFPGAN made local discriminators on the left eye, right eye, and mouth popular [3]. We add a nose region, which we find reduces mid-face artifacts in small generators.

C. Iterative and recurrent restoration

Iterative and recurrent designs are common in super-resolution. Back-projection networks [9] alternate up- and down-projection blocks, DIC [10] refines face detail in a recurrent way, and some methods reapply a small generator several times [11]. Diffusion-based restorers [23], [24] and direct-iteration models [25] also treat restoration as repeated steps. The closest work to ours [11] trains a small network with a few recurrent passes, but it supervises only the final output and does not treat G as a fixed-point operator. CFRNet is different in two ways: we supervise every cycle with increasing weights, and we add an idempotence loss and a re-degradation cycle loss that tie the procedure to a fixed point.

D. Fixed-point and cycle objectives

Deep equilibrium models [26] and consistency models [27] use fixed-point ideas, but they usually need an iterative solver in the forward pass and are heavy. Cycle consistency is widely used in unpaired translation [28], and re-degradation cycles appear in recent reference-based face restoration [29] and unsupervised diffusion-prior restoration [30]. To our knowledge, no prior face-restoration work combines a small feed-forward CNN, weight-shared multi-cycle inference, and joint fixed-point and re-degradation training.

E. The perception-distortion trade-off

Blau and Michaeli [31] show that pixel distortion (such as PSNR) and perceptual quality (such as LPIPS or FID) cannot both be minimized at once. Later face-restoration work puts perceptual metrics first. Our cycle analysis (Section IV-D) shows this trade-off inside one trained CFRNet: the cycle that is best for PSNR is not the cycle that is best for LPIPS.

F. Efficient networks for on-device restoration

On-device restoration usually uses small backbones and depthwise-separable operators [7], [32], [33], [8]. CFRNet uses a simple ResNet-style encoder-decoder with standard convolutions, ReLU, and nearest-neighbor upsampling, which are all supported by embedded NPU toolchains and quantize to INT8 [34] without much loss.

III. METHOD

A. Overview

Given a degraded face $x \in \mathbb{R}^{H \times W \times 3}$ and its clean version y , we want a small generator G such that running G a few times gives a good restoration:

$$\hat{y} = G^{(k)}(x) := \underbrace{G \circ G \circ \dots \circ G}_{k \text{ times}}(x). \quad (1)$$

Fig. 1 shows CFRNet. It has three parts: a small generator (Section III-B), component-aware local supervision (Section III-C), and CCFP training (Section III-D).

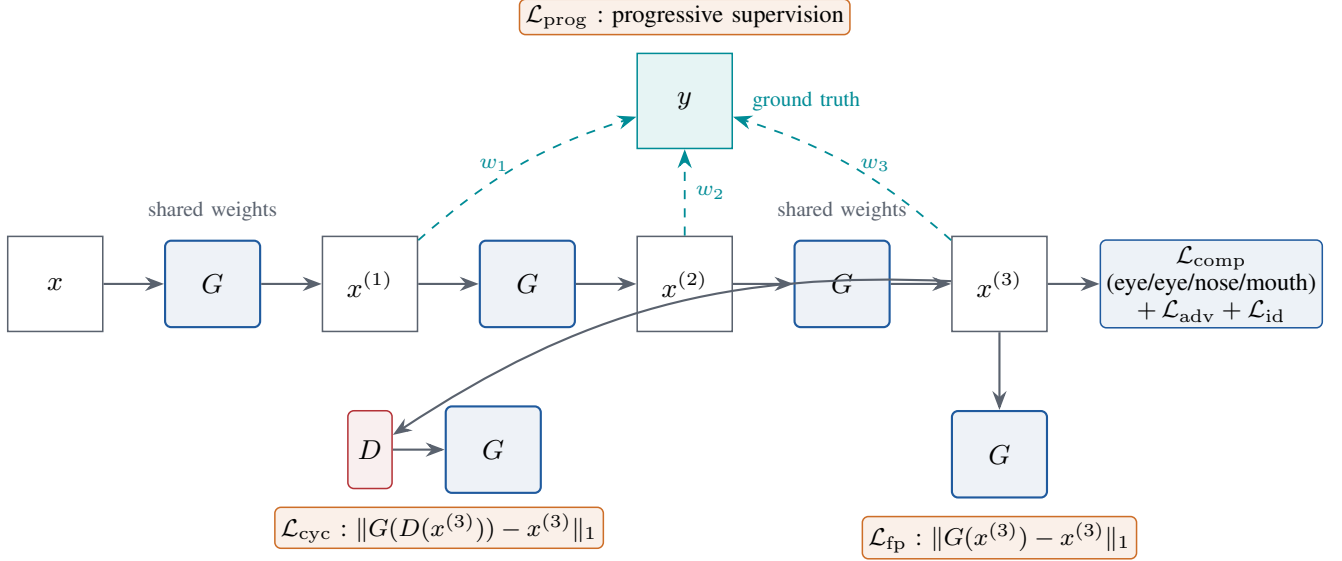


Fig. 1. CFRNet and CCFP training. The same 2.0M-parameter generator G is run $k = 3$ times with shared weights. Progressive supervision compares each cycle output $x^{(i)}$ to the ground truth y with increasing weights $w_1 < w_2 < w_3$. The idempotence loss pushes the last output toward a fixed point of G . The re-degradation cycle loss degrades the restored image with D and restores it again, and asks the result to match the original restoration. Local discriminators on the eyes, nose, and mouth, together with adversarial and identity losses, supervise the final cycle. At inference only the top row runs; no extra parameters or operators are added.

TABLE I
GENERATOR CONFIGURATION ($H = W = 256$). TOTAL: **2.00 M**
PARAMETERS, **8.7 G** MACS PER CYCLE.

Stage	Operator	Output size	Channels
Input	–	256×256	3
Stem	3×3 conv	256×256	32
Down-1	3×3 conv, $s=2$	128×128	64
Down-2	3×3 conv, $s=2$	64×64	128
Body	ResBlock $\times 6$	64×64	128
Up-1	conv + nn-up	128×128	64
Up-2	conv + nn-up	256×256	32
Head	3×3 conv + residual add + tanh	256×256	3

B. Small Generator

The generator is a ResNet-style encoder-decoder (Table I) with base width $C = 32$, two stride-2 downsampling stages, $N = 6$ residual blocks at the bottleneck, and two upsampling stages that use nearest-neighbor upsampling with 3×3 convolutions. Skip connections join matching encoder and decoder stages. The output head predicts a residual that is added to the input before a tanh: $\hat{y} = \tanh(x + \text{head}(\cdot))$. The residual design is important for stable training. With small initial head weights the network starts near the identity, which avoids the constant-output collapse we saw with direct image regression. The backbone uses only convolution, ReLU, residual addition, and nearest-neighbor upsampling. All of these are supported by embedded NPU toolchains and quantize to INT8 cleanly.

C. Component-Aware Supervision

Following GFPGAN [3], we put local discriminators on aligned face-component patches. We add a nose region to the GFPGAN set:

$$\mathcal{R} = \{\text{le, re, n, m}\}. \quad (2)$$

For each region r , ROI Align crops the restored patch $\hat{y}_r$ and the ground-truth patch y_r , and a small local discriminator D_r is trained with them. The component loss is a local hinge adversarial term plus a feature-style (Gram) term:

$$\mathcal{L}_{\text{comp}} = \sum_{r \in \mathcal{R}} \left[-\lambda_{\text{loc}} \mathbb{E}_{\hat{y}_r} [D_r(\hat{y}_r)] + \lambda_{\text{fs}} \|\text{Gram}(\psi_r(\hat{y}_r)) - \text{Gram}(\psi_r(y_r))\|_1 \right], \quad (3)$$

where ψ_r are features from D_r . The discriminators use spectral normalization. The nose region reduces ringing around the nostrils and the nasal bridge, which is a common failure of small CNN restorers.

D. Cycle-Consistent Fixed-Point Training

1) *Motivation:* A single-pass loss $\mathcal{L}(G(x), y)$ learns the map from a degraded image to a clean one. It says nothing about how G behaves on its own outputs. So when we run G again at test time, two things can go wrong: the second pass can over-sharpen or add fake texture because $G(x^{(1)})$ is out of distribution, or the iterations can oscillate and never settle. CCFP treats G as a discrete dynamical system on image space and trains it to move toward the set of natural faces $\mathcal{M}_y$ and to stop there (Fig. 2). The clean image y is the target fixed point, and we want each trajectory $\{x^{(0)} = x, x^{(1)}, x^{(2)}, \dots\}$ to converge near $\mathcal{M}_y$ and stay.

2) *The three CCFP losses:* Let $x^{(0)} = x$ and $x^{(i)} = G(x^{(i-1)})$ for $i = 1, \dots, k$. We use $k = 3$.

a) *Progressive multi-cycle supervision:* Each cycle output is supervised against y with increasing weight:

$$\mathcal{L}_{\text{prog}} = \sum_{i=1}^k w_i [\|x^{(i)} - y\|_1 + \lambda_{\text{per}} \|\phi(x^{(i)}) - \phi(y)\|_1], \quad (4)$$

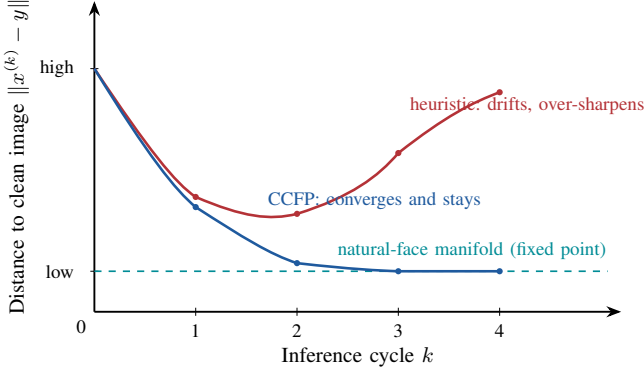


Fig. 2. Distance to the clean image as a function of the cycle count. A single-pass network reused by hand (red) improves for one or two passes and then drifts back up, because its own outputs are out of distribution. CCFP (blue) keeps the distance going down and then flat, which is the fixed-point behavior. The matching numbers are in Table V.

where ϕ is a pretrained VGG-19 feature extractor [35] and $w_1 = 0.3$, $w_2 = 0.5$, $w_3 = 1.0$. The later cycle gets more weight because it is the deployment output, and the earlier cycles are still supervised so that each pass gives a usable image. This loss replaces the usual single-pass reconstruction loss.

b) *Idempotence loss*: The last cycle output should be a fixed point of G :

$$\mathcal{L}_{\text{fp}} = \|G(x^{(k)}) - x^{(k)}\|_1. \quad (5)$$

This costs one extra forward pass at training time. It says that one more pass should not change the output, which prevents over-sharpening and gives a natural stopping point. In other words, it makes $\mathcal{M}_y$ a stable attractor of G .

c) *Re-degradation cycle loss*: Let D be a random degradation simulator that follows the training degradation. Degrading a restored image and restoring it again should give the same restoration:

$$\mathcal{L}_{\text{cyc}} = \|G(D(x^{(k)})) - x^{(k)}\|_1. \quad (6)$$

Where $\mathcal{L}_{\text{prog}}$ teaches G to map x to y , this loss teaches G to undo the degradation applied to a clean image. Because D is random, this term also works as a regularizer and helps on harder degradations.

3) *Full objective*: The full training loss is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{prog}} + \lambda_{\text{fp}} \mathcal{L}_{\text{fp}} + \lambda_{\text{cyc}} \mathcal{L}_{\text{cyc}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \mathcal{L}_{\text{comp}} + \lambda_{\text{id}} \mathcal{L}_{\text{id}}, \quad (7)$$

where $\mathcal{L}_{\text{adv}}$ is a global hinge GAN loss on $x^{(k)}$ only, $\mathcal{L}_{\text{id}}$ is an ArcFace identity loss [36], and $\mathcal{L}_{\text{comp}}$ is the component loss above. We use $\lambda_{\text{per}} = 1$, $\lambda_{\text{fp}} = 0.5$, $\lambda_{\text{cyc}} = 0.5$, $\lambda_{\text{adv}} = 0.1$, $\lambda_{\text{loc}} = 1$, $\lambda_{\text{fs}} = 1$, $\lambda_{\text{id}} = 10$.

4) *Training stability*: We raise λ_{fp} from 0 to 0.5 over the first 100K iterations. If it is large from the start, the idempotence loss can collapse G to the identity, which trivially gives $G(G(x)) = G(x)$. We also use a 5,000-iteration GAN warm-up, where only the reconstruction terms drive training, followed by a 5,000-iteration ramp on the adversarial and component-discriminator losses. The discriminators use spectral normalization and a hinge loss, and we clip the gradient

Algorithm 1 One CCFP training step

Require: clean image y , degradation D , generator G , cycles $k = 3$

- 1: $x \leftarrow D(y)$ // sample a degraded input
- 2: $x^{(0)} \leftarrow x$
- 3: **for** $i = 1$ to k **do**
- 4: $x^{(i)} \leftarrow G(x^{(i-1)})$
- 5: **end for**
- 6: $x^{(\text{fp})} \leftarrow G(x^{(k)})$ // for the idempotence loss
- 7: $x^{(\text{cyc})} \leftarrow G(D(x^{(k)}))$ // for the cycle loss
- 8: **compute** $\mathcal{L}_{\text{prog}}$ from $\{x^{(i)}\}$ and y
- 9: $\mathcal{L}_{\text{fp}} \leftarrow \|x^{(\text{fp})} - x^{(k)}\|_1$
- 10: $\mathcal{L}_{\text{cyc}} \leftarrow \|x^{(\text{cyc})} - x^{(k)}\|_1$
- 11: **compute** $\mathcal{L}_{\text{adv}}, \mathcal{L}_{\text{comp}}, \mathcal{L}_{\text{id}}$ from $x^{(k)}$ and y
- 12: **backprop** $\mathcal{L}_{\text{total}}$ and update G

norm at 1.0. These choices were needed: without them, early runs collapsed to a constant image.

5) *Inference*: At test time we run G for k cycles with shared weights, $\hat{y} = G^{(k)}(x)$. We do not use the idempotence pass or the degradation simulator at test time, since both are training-only. Latency grows linearly with k . We study the effect of k in Section IV-D and use $k = 3$ by default.

IV. EXPERIMENTS

A. Setup

a) *Training data*: We train on FFHQ [37], which has 70K aligned high-quality faces. Images are resized to 256×256 . We use a fixed 90/5/5% split: 63K for training, 3.5K for validation, and 3.5K held out.

b) *Degradation D* : Following GFPGAN [3], [38], D applies four steps with random parameters: Gaussian blur with $\sigma \in [1, 15]$; bicubic downsampling by a random scale $r \in [1, 12]$ and bicubic upsampling back to 256×256 ; additive Gaussian noise with $\sigma_n \in [0, 20]$ on the $[0, 255]$ scale; and JPEG compression with quality $q \in [30, 90]$. The same D is used for the re-degradation cycle loss. For evaluation we fix the random seeds per image, so every method sees the same degraded inputs.

c) *Evaluation set*: We use a fixed test set of 300 FFHQ images that are held out from training. The same per-image degradation is applied to all methods, so the inputs are identical across the methods we compare. We report PSNR, SSIM, and LPIPS averaged over the 300 images.

d) *Baselines*: For a fair comparison under our deployment limits (256×256 crops, INT8 on the Hi3402, no pretrained prior), we reimplement each method as a deployment-compatible model trained from scratch on the same data:

- **GFPGAN-Lite** (~17M): a U-Net restorer that follows GFPGAN’s degradation-removal module [3] but drops the frozen StyleGAN2 prior and the SFT modules, which are not NPU-compilable. Trained with the same losses as GFPGAN (L1, perceptual, adversarial, identity, and eye/mouth component discriminators).
- **GPEN-Lite** (~15M): a style-modulated U-Net inspired by GPEN [4], without GPEN’s full StyleGAN2 decoder.

Trained with reconstruction, adversarial, and identity losses.

- **CodeFormer-Lite** (~ 10.5 M): a VQ-VAE-style restorer with a 1024-entry codebook, trained end to end with reconstruction, adversarial, and commitment losses. It drops the two-stage codebook pretraining and the transformer code prediction of the official CodeFormer [6], which do not compile on the NPU.

This setup separates the effect of each design’s architecture from the effect of large pretrained priors that cannot run on the target NPU. We do not compare against the full 512×512 official models, except for the qualitative parity note in Section IV-B.

e) Metrics: We report PSNR and SSIM for distortion and LPIPS [39] for perceptual quality. For blind face restoration the perceptual metric is the main one [31]; PSNR and SSIM are given for completeness and to show the perception-distortion trade-off (Section IV-D).

f) Implementation: Generator: $C = 32$, $N = 6$ (Table I). Global discriminator: a PatchGAN [40] with spectral normalization [41]. Component discriminators: 4-layer SN-Conv-LeakyReLU networks on ROI crops (40×40 for eyes and nose, 80×50 for the mouth). We use Adam ($\text{lr} = 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$) and batch size 16. CFRNet is trained for 32 epochs at the base learning rate and then fine-tuned with the learning rate scaled by 0.25. All baselines use the same hardware (one NVIDIA L40S) and the same number of iterations.

g) Latency measurement: We report latency on two platforms. The server number is measured on one NVIDIA Tesla V100 in FP32 with batch size 1, using CUDA events and 50 timed runs after 10 warm-up runs. The board number is measured on the HiSilicon Hi3402 NPU in INT8. The two numbers come from different hardware and precision, so they are not directly comparable; we give both to be clear about what runs where.

B. Main Comparison

Table II reports the comparison under the shared protocol. The degraded input has PSNR 22.45 and LPIPS 0.712 as a reference floor.

There are three main points. First, CFRNet gives the best perceptual score by a clear margin. Its LPIPS at $k=3$ (0.250) is 29% lower than GFPGAN-Lite (0.353), 34% lower than GPEN-Lite, and 35% lower than CodeFormer-Lite, with 5 to 8 times fewer parameters. Because LPIPS is the main metric for this task [31], [3], [6], this is the headline result. CFRNet at $k=2$ also gives the best PSNR (23.82) and the best SSIM (0.685), so it is not trading away pixel fidelity to get there. Second, CFRNet is the only model that runs on the NPU. The Lite baselines still keep operators that the Hi3402 INT8 toolchain cannot compile, even after we removed the heavy prior modules. CFRNet uses only standard CNN operators and runs end to end at 69 ms for $k=3$. Third, PSNR and SSIM behave differently from LPIPS across cycles. CFRNet’s PSNR is highest at $k=2$ (23.82) and lower at $k=3$ (23.10) and $k=4$ (22.98). This is not a bug. It is the known perception-distortion

trade-off [31] inside one network: the adversarial and identity losses act only on the last cycle and push it toward a sharper, more realistic image, which lowers pixel fidelity a little. We look at this next.

a) Parity with the full GFPGAN: The comparison above uses only deploy-compatible from-scratch baselines, since no 512×512 prior-guided model compiles on our NPU. For context, we also ran the full official GFPGAN, with its frozen StyleGAN2 prior at its native resolution, on the same inputs. CFRNet at $k=3$ gives about the same visual quality as this much larger model (see also Section IV-H), while using a small fraction of its parameters and compute and, unlike it, running on the Hi3402. We report this as a qualitative observation, not a benchmark number, because the two models work at different native resolutions. Still, reaching the quality of GFPGAN’s pretrained-prior pipeline with a 2.0 M-parameter standard-CNN restorer is a useful result for deployment.

C. Qualitative Results

Fig. 3 compares CFRNet ($k=3$) with the retrained Lite baselines. Fig. 4 shows the cycle-by-cycle output of one model and makes the fixed-point behavior easy to see: the output stops changing once it reaches a natural face. Fig. 5 zooms into the eye and mouth, where small baselines fail most often, and shows that CFRNet follows the ground truth more closely.

D. Effect of the Cycle Count

Table III lists the metrics and the latency on both platforms across cycles.

a) Why the curves differ: The adversarial and identity losses act only on the final cycle $x^{(k)}$. The earlier cycles see only the reconstruction terms. So the network spreads the work out: the first two cycles mostly denoise, which helps PSNR, and the third cycle adds the adversarial and identity detail, which helps LPIPS at a small cost in PSNR. This is the perception-distortion trade-off [31] seen inside one model.

b) How to choose k : The best cycle count depends on the use case. Use $k=3$ for the best perceptual quality, for example photo and video apps where a person looks at the output. Use $k=2$ for the best PSNR and SSIM, for example when the restored image feeds a later pixel-level step. Use $k=1$ for the lowest latency on a very tight budget. Because k is chosen at inference, no retraining or weight reloading is needed to move along this curve.

c) Fixed-point check: Since the model was trained with $k_{\text{train}} = 3$, the $k=4$ result runs G once more than training did. The output barely moves: LPIPS rises by only 0.012 and PSNR drops by only 0.12 dB. This means $x^{(3)}$ is close to a fixed point of G . A single-pass model behaves differently and drifts at $k > k_{\text{train}}$ (the red curve in Fig. 2), as the next section shows.

E. Ablation: CCFP Losses

Table IV isolates each CCFP loss. All rows use the same generator and run at $k=3$; they differ only in the training loss.

TABLE II

COMPARISON ON THE 300-IMAGE FFHQ-256 TEST SET. ALL BASELINES ARE REIMPLEMENTED AS DEPLOY-COMPATIBLE “LITE” MODELS AND TRAINED FROM SCRATCH AT 256×256 WITH THE SAME DEGRADATION, SCHEDULE, AND HARDWARE AS CFRNET. “BOARD” LATENCY IS ON THE HISILICON HI3402 NPU IN INT8; *infeasible* MEANS THE MODEL CANNOT BE COMPILED TO THE NPU. “SERVER” LATENCY IS ON ONE V100 IN FP32. BEST IN **BOLD**, SECOND UNDERLINED. CFRNET VARIANTS SHARE THE SAME 2.0 M PARAMETERS; ONLY THE CYCLE COUNT DIFFERS.

Method	Params ↓	MACs/pass ↓	PSNR ↑	SSIM ↑	LPIPS ↓	Server FP32 (ms)	Board INT8 (ms)
Input (degraded)	–	–	22.45	0.621	0.712	–	–
GFPGAN-Lite [3]	~17.0 M	~23 G	23.49	0.668	0.353	–	infeasible
GPEN-Lite [4]	~15.0 M	~19 G	23.18	0.659	0.378	–	infeasible
CodeFormer-Lite [6]	~10.5 M	~11 G	22.71	0.642	0.382	–	infeasible
CFRNet ($k=1$)	2.0 M	8.7 G	23.54	0.673	0.360	2.7	23
CFRNet ($k=2$)	2.0 M	17.4 G	23.82	0.685	0.300	4.9	46
CFRNet ($k=3$, default)	2.0 M	26.1 G	23.10	0.662	0.250	7.3	69
CFRNet ($k=4$)	2.0 M	34.8 G	22.98	0.668	<u>0.262</u>	9.6	92

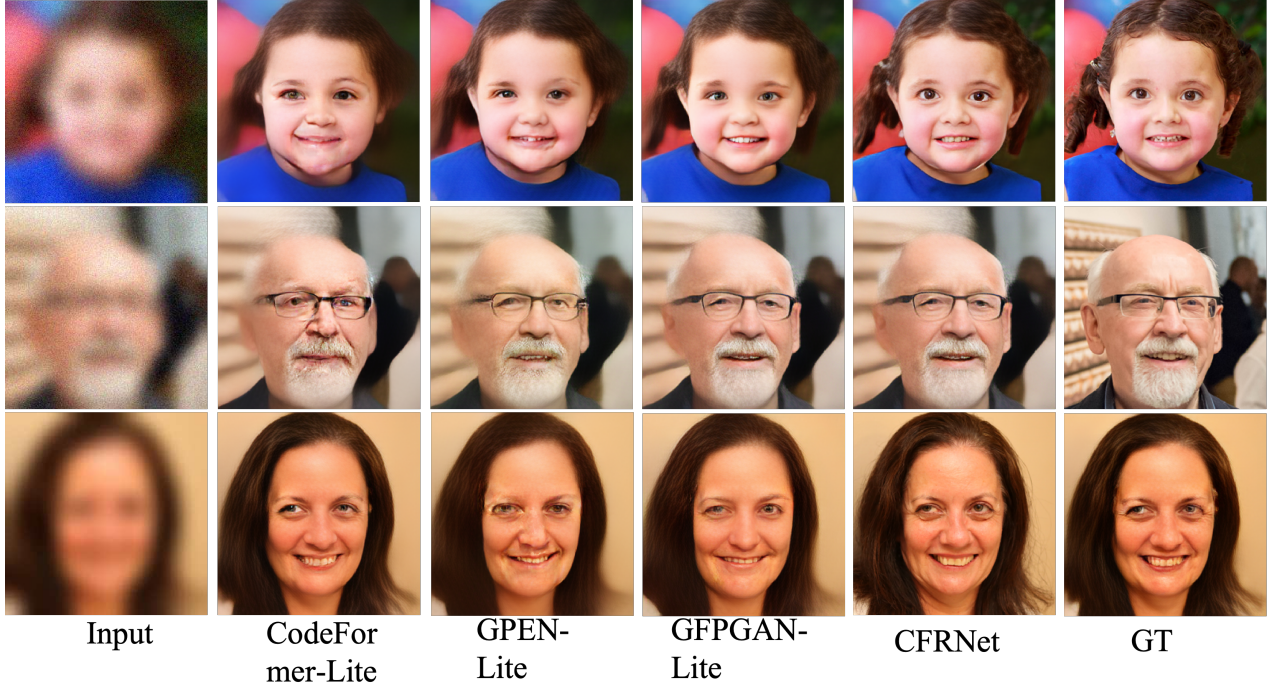


Fig. 3. Qualitative comparison on the FFHQ-256 test set. Left to right: degraded input, CodeFormer-Lite, GPEN-Lite, GFPGAN-Lite, CFRNet ($k=3$), ground truth. All baselines are trained from scratch under the same protocol. Across a child, an older man with glasses, and an adult, CFRNet gives comparable or finer detail in the eyes, hair, and skin with 5 to 8 times fewer parameters, and with fewer mid-face artifacts than the Lite baselines. The glasses frames and beard in the second row, which are hard for small restorers, stay sharp and free of ringing.

TABLE III

EFFECT OF THE CYCLE COUNT FOR ONE TRAINED CFRNET. PSNR IS BEST AT $k=2$, LPIPS IS BEST AT $k=3$, AND BOTH DROP ONLY A LITTLE AT $k=4$. THE MODEL WAS TRAINED WITH $k_{\text{train}} = 3$, SO $k=4$ IS ONE PASS BEYOND TRAINING, YET THE CHANGE IS SMALL. SERVER LATENCY IS V100 FP32; BOARD LATENCY IS HI3402 INT8.

k	PSNR ↑	SSIM ↑	LPIPS ↓	Server (ms)	Board (ms)
1	23.54	0.673	0.360	2.7	23
2	23.82	0.685	0.300	4.9	46
3	23.10	0.662	0.250	7.3	69
4	22.98	0.668	0.262	9.6	92

TABLE IV

ABLATION OF THE CCFP LOSSES AT $k=3$. “SINGLE-PASS” IS STANDARD ONE-PASS TRAINING REUSED $k=3$ TIMES AT TEST TIME, THE SETTING USED BY EARLIER RECURRENT WORK [11]. EACH ROW ADDS THE LISTED LOSS TO THE ROW ABOVE.

Training setting	PSNR ↑	LPIPS ↓	SSIM ↑
Single-pass (heuristic $k=3$)	22.62	0.330	0.636
+ progressive $\mathcal{L}_{\text{prog}}$	22.88	0.292	0.650
+ idempotence $\mathcal{L}_{\text{fp}}$	23.02	0.271	0.657
+ cycle $\mathcal{L}_{\text{cyc}}$ (full CCFP)	23.10	0.250	0.662
gain over single-pass	+0.48	−0.080	+0.026

Each loss helps. Progressive supervision gives the largest single LPIPS drop (−0.038) by removing the train-test mismatch. The idempotence loss adds another −0.021 and makes

the cycles stable, which we check below. The cycle loss adds another −0.021, mostly by improving results on harder degradations.

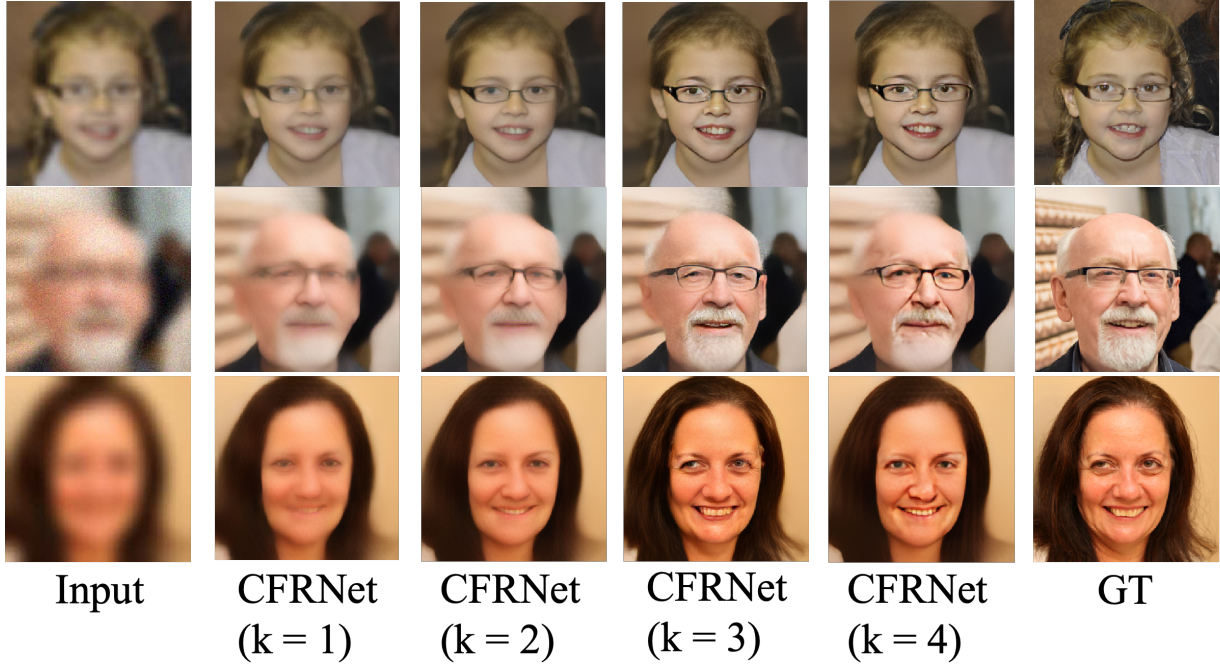


Fig. 4. Cycle-by-cycle output of one trained CFRNet. Left to right: degraded input, $k=1$, $k=2$, $k=3$, $k=4$, ground truth. Every cycle reuses the same 2.0M weights. Early cycles mostly denoise; later cycles sharpen the face and restore detail in the eyes, mouth, and hair. The $k=3$ and $k=4$ outputs look almost the same, which is the fixed-point behavior from CCFP and matches Tables III and V.

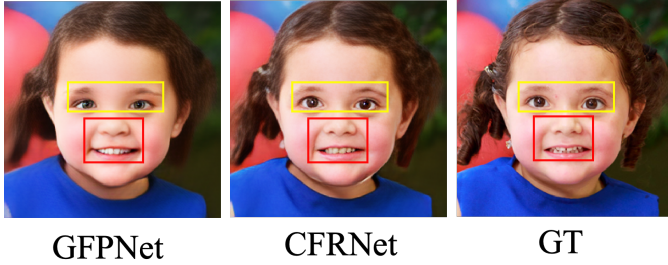


Fig. 5. Component-level comparison with zoom-ins. Yellow box: eye region; red box: mouth region. Left to right: GFPNet (labeled GFPNet in the panel), CFRNet ($k=3$), ground truth. CFRNet restores the eye and the teeth and mouth closer to the ground truth, while the small GFPNet baseline tends to over-smooth the iris and blur the teeth. The mouth result reflects the added nose and mouth supervision (Section III-C); the matching numbers are in Table VI.

TABLE V
PER-CYCLE CHANGE Δ_k (MEAN L_2 OVER THE TEST SET; SMALLER AT LARGE k IS MORE FIXED-POINT-LIKE). CCFP CONVERGES MUCH MORE TIGHTLY THAN THE SINGLE-PASS MODEL.

Setting	Δ_2	Δ_3	Δ_4
Single-pass training	0.071	0.058	0.052
Full CCFP	0.043	0.022	0.018

a) *Checking the fixed-point property:* To check that CCFP gives a contractive operator, we measure the per-cycle change $\Delta_k = \|G^{(k+1)}(x) - G^{(k)}(x)\|_2$ on the test set.

The single-pass model still changes a lot at $k=4$, while CCFP cuts Δ_4 by about 65%. This shows that CCFP makes G settle near $\mathcal{M}_y$, which matches Fig. 2.

TABLE VI
EFFECT OF THE FACE-COMPONENT DISCRIMINATORS (CFRNet AT $k=3$, FULL CCFP). EACH ROW ADDS THE LISTED SUPERVISION TO THE ROW ABOVE.

Training objective	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
No component losses	22.90	0.655	0.285
+ eye/mouth (GFPNet-style [3])	23.02	0.659	0.266
+ eye/mouth/nose (full)	23.10	0.662	0.250

TABLE VII
DEPLOYMENT ON THE HISILICON HI3402 NPU. THE INT8 TOOLCHAIN COMPILES CFRNet END TO END BUT CANNOT COMPILE THE OPERATOR SETS THAT THE LITE BASELINES STILL USE, SUCH AS THE CODEBOOK LOOKUP IN CODEFORMER-LITE AND THE STYLE MODULATION IN GPEN-LITE.

Item	GFPNet-Lite	GPEN-Lite	CodeFormer-Lite	CFRNet
Input size	256×256	256×256	256×256	256×256
Params (M) $\downarrow$	~17	~15	~10	2.0
MACs per pass (G)	~23	~19	~11	8.7
INT8 quantization	partial*	no [†]	no [‡]	yes
Hi3402 compilable	no	no	no	yes
Board latency (ms)	infeasible	infeasible	infeasible	23 ($k=1$) / 69 ($k=3$)

* U-Net skip-concatenation shapes need an FP16 fallback. [†] Style-modulation ops are not in the Hi3402 INT8 op set. [‡] Codebook nearest-neighbor lookup is not a standard CNN op.

F. Ablation: Component Losses and the Nose Region

Adding the nose region gives another -0.016 LPIPS over the eye-and-mouth setting. In the images (Fig. 3 and Fig. 5) the nose region removes ringing around the nostrils and the nasal bridge, a common problem for small CNN restorers.



Fig. 6. The plain variant on a hard real-world input (heavy haze, low contrast, far from the FFHQ training data). Left to right: degraded input, CFRNet-D direct-1 (one pass), CFRNet-D direct-2 (the same plain network run twice), ground truth, and GFPNet-Lite (labeled GFPNet). One pass already removes most of the degradation. Because the second pass sees a cleaner input, it recovers more detail and reaches about the quality of the full CCFP model at $k=3$, and it is better than the small GFPNet-Lite baseline on this input. On such inputs we also found CFRNet-D direct-2 to be about the same as the full official GFPNet, here with a network small enough to run at 23 ms per pass on the board.

G. Deployment on the Consumer NPU

On the Hi3402, CFRNet finishes a 3-cycle restoration in about 69 ms at 256×256 , or about 23 ms per cycle. For comparison, the same model runs in about 2.4 ms per cycle on a V100 in FP32 (Table III). The gap is expected, since the V100 is a large datacenter GPU and the Hi3402 is a small embedded NPU; both numbers describe the same model at the two ends of the deployment range. The board latency leaves room for face detection, alignment, and rendering inside a real-time budget on a device that cannot run any of the baselines we tested.

H. A Simpler Variant: A Plain Iterated CNN

CCFP gives our best perceptual quality, but it has a cost at training time. The multi-cycle unroll plus the idempotence and cycle passes make training take about twice as long as standard single-pass training. For teams that want the simplest possible pipeline, we asked whether the main idea, that running a small restorer again moves it toward a natural face, still works in a much simpler setting. So we trained a separate, plain CNN, which we call CFRNet-D, with the standard single-pass recipe (no progressive unroll, no idempotence, no cycle loss), and at test time we just fed its output back in. We write direct- j for j passes. This model is very easy to train and to deploy: one static graph, one operator set, and refinement by simply running the same graph again, at about 23 ms per pass on the board.

Fig. 6 shows a hard real-world example. One pass (direct-1) removes most of the degradation, and the second pass (direct-2) sees a cleaner input and recovers more detail, reaching about the quality of the full CCFP model at $k=3$ and beating our GFPNet-Lite baseline on this input. We do not present CFRNet-D as a replacement for CCFP. CCFP is the principled, best-quality model with a built-in fixed-point property and a no-retrain quality knob. CFRNet-D simply shows that the iterative idea is robust enough to work even with a very plain, easy-to-ship network. Two points follow. First, parameter count and architectural complexity are not the only things that set restoration quality; how inference is structured can matter as much as model size. Second, for consumer products where integration cost often matters more than training cost,

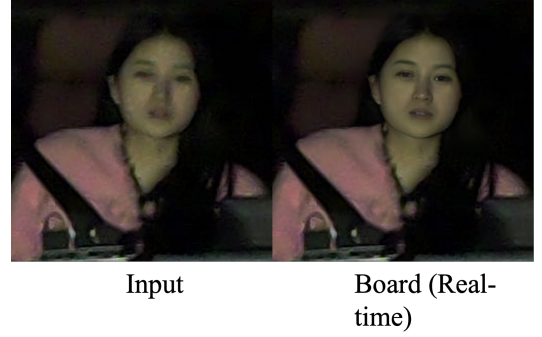


Fig. 7. Real-time output on the embedded board in a driver-monitoring setting. Left: raw on-device input (low light, motion, strong shadows, far from the FFHQ training data). Right: the board's real-time output. The model restores the face and skin tone directly from the on-device stream inside the per-frame budget, with no offline pre- or post-processing. This is the setting we ship.

a plain iterated CNN can be a good practical choice. We think inference-time structure deserves more attention when restoration models go to limited hardware.

I. Real-Time In-Car Deployment

Fig. 7 shows a frame captured and restored in real time on the board in a driver-monitoring setting, which is low light and motion-heavy and far from the FFHQ training data. The model restores the face and skin tone directly from the on-device stream inside the per-frame budget, with no offline steps. This in-the-wild result on inputs that differ from the training data is helped by the re-degradation cycle loss and by the stability that the fixed-point training gives.

V. DISCUSSION AND LIMITATIONS

a) What CCFP does: A single-pass G is trained only on degraded inputs $\mathcal{X}$. Its outputs $G(\mathcal{X})$ are close to, but not on, the natural-face set $\mathcal{M}_y$. Running G again on $G(x)$ is therefore out of distribution, and the result is not controlled, so it can over-sharpen, drift in color, or oscillate (the red curve in Fig. 2). CCFP fixes this by training G on its own outputs (progressive supervision), by penalizing further change at convergence (idempotence), and by encoding the degradation process (re-degradation cycle).

b) Why PSNR peaks at $k=2$: This is the perception-distortion trade-off [31] inside one network. The adversarial and identity losses act only on the last cycle, so the network uses that cycle to add realistic detail, which helps LPIPS but slightly lowers PSNR. For deployment this is fine, because the task is perceptual and a person looks at the output. The choice between $k=2$ and $k=3$ gives the user a simple, no-retrain knob along the trade-off.

c) Limitations: First, latency grows linearly with k . On the Hi3402 $k=3$ is fast enough, but a tighter chip may prefer $k=1$ or $k=2$, with the quality shown in Section IV-D. Second, the re-degradation cycle loss assumes the deployment degradation is similar to the training degradation D . A very different degradation, such as extreme low light or motion blur that D does not cover, may not be handled, although the in-car result in Section IV-I is encouraging. Third, CCFP training

takes about twice as long as single-pass training, because each step runs G several times plus the idempotence and cycle passes. This is a one-time cost and does not affect inference, and the plain CFRNet-D variant (Section IV-H) trades a small quality margin for standard training cost. Fourth, our numbers are against from-scratch Lite reimplementations of GFPGAN, GPEN, and CodeFormer, which is the right setting for on-device use; the parity note against the full official GFPGAN is a qualitative observation at a different native resolution, not a benchmark number.

VI. CONCLUSION

We presented CFRNet, a small blind face restoration model for consumer NPUs, and CCFP, its training method. CCFP closes a gap in iterative restoration: instead of training a network for one mapping and reusing it by hand, we train it as a fixed-point operator on the set of natural faces, using progressive supervision, an idempotence loss, and a re-degradation cycle loss. With a 2.0M-parameter NPU-friendly generator and component supervision that adds a nose region, CFRNet gives the best perceptual score among comparable from-scratch baselines on a 300-image test set, gives the best PSNR and SSIM at $k=2$, and is the only model that runs on the target NPU. It also reaches about the quality of the full official GFPGAN. The cycle count is a simple, no-retrain knob between pixel fidelity ($k=2$) and perceptual quality ($k=3$). Finally, the same idea works with a plain, easy-to-ship CNN that reaches $k=3$ quality on hard inputs, and the model runs in real time on an in-car driver-monitoring board. Future work includes extending CCFP to 512×512 deployments, testing on larger real-world sets, and learning a per-image stopping rule for the cycles.

ACKNOWLEDGMENT

The authors thank the reviewers for their comments. The source code will be made publicly available upon acceptance of this manuscript. The authors also acknowledge the use of large language models (Google Gemini) strictly for English language editing and proofreading during the preparation of this manuscript.

REFERENCES

- [1] B. Reddy, Y.-H. Kim, S. Yun, C. Seo, and J. Jang, "Real-time driver drowsiness detection for embedded system using model compression of deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2017, pp. 438–445.
- [2] J. Yu, S. Park, S. Lee, and M. Jeon, "Driver drowsiness detection using condition-adaptive representation learning framework," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 11, pp. 4206–4218, 2019.
- [3] X. Wang, Y. Li, H. Zhang, and Y. Shan, "Towards real-world blind face restoration with generative facial prior," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9168–9178.
- [4] T. Yang, P. Ren, X. Xie, and L. Zhang, "Gan prior embedded network for blind face restoration in the wild," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 672–681.
- [5] Z. Wang, J. Zhang, R. Chen, W. Wang, and P. Luo, "Restoreformer: High-quality blind face restoration from undegraded key-value pairs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17 512–17 521.
- [6] S. Zhou, K. C. Chan, C. Li, and C. C. Loy, "Towards robust blind face restoration with codebook lookup transformer," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 30 599–30 611.
- [7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [8] J. Liu, J. Tang, and G. Wu, "Residual feature distillation network for lightweight image super-resolution," in *European Conference on Computer Vision (ECCV) Workshops*, 2020, pp. 41–55.
- [9] M. Haris, G. Shakhnarovich, and N. Ukita, "Deep back-projection networks for super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 1664–1673.
- [10] C. Ma, Z. Jiang, Y. Rao, J. Lu, and J. Zhou, "Deep face super-resolution with iterative collaboration between attentive recovery and landmark estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5569–5578.
- [11] Z. Li, J. Yang, Z. Liu, X. Yang, G. Jeon, and W. Wu, "Feedback network for image super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3867–3876.
- [12] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 38, no. 2, pp. 295–307, 2016.
- [13] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2017, pp. 136–144.
- [14] Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong, and Y. Fu, "Image super-resolution using very deep residual channel attention networks," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 286–301.
- [15] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "Swinir: Image restoration using swin transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2021, pp. 1833–1844.
- [16] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2672–2680.
- [17] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4681–4690.
- [18] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. C. Loy, "Esrgan: Enhanced super-resolution generative adversarial networks," in *European Conference on Computer Vision (ECCV) Workshops*, 2018, pp. 63–79.
- [19] A. Bulat and G. Tzimiropoulos, "Super-fan: Integrated facial landmark localization and super-resolution of real-world low resolution faces in arbitrary poses with gans," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 109–117.
- [20] Y. Chen, Y. Tai, X. Liu, C. Shen, and J. Yang, "Fsnet: End-to-end learning face super-resolution with facial priors," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2492–2501.
- [21] Y. Gu, X. Wang, L. Xie, C. Dong, G. Li, Y. Shan, and M.-M. Cheng, "Vqfr: Blind face restoration with vector-quantized dictionary and parallel decoder," in *European Conference on Computer Vision (ECCV)*, 2022, pp. 126–143.
- [22] X. Li, C. Chen, S. Zhou, X. Lin, W. Zuo, and L. Zhang, "Blind face restoration via deep multi-scale component dictionaries," in *European Conference on Computer Vision (ECCV)*, 2020, pp. 399–415.
- [23] C. Saharia, J. Ho, W. Chan, T. Salimans, D. J. Fleet, and M. Norouzi, "Image super-resolution via iterative refinement," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 45, no. 4, pp. 4713–4726, 2023.
- [24] Z. Yue and C. C. Loy, "Difface: Blind face restoration with diffused error contraction," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 46, no. 12, pp. 9991–10 004, 2024.

- [25] M. Delbracio and P. Milanfar, "Inversion by direct iteration: An alternative to denoising diffusion for image restoration," *Transactions on Machine Learning Research (TMLR)*, 2023.
- [26] S. Bai, J. Z. Kolter, and V. Koltun, "Deep equilibrium models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [27] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," in *International Conference on Machine Learning (ICML)*, 2023, pp. 32 211–32 252.
- [28] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2223–2232.
- [29] Z. Yin, J. Chen, M. Liu, Z. Wang, F. Li, R. Pei, X. Li, R. W. Lau, and W. Zuo, "Refstar: Blind face image restoration with reference selection, transfer, and reconstruction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 14, 2026, pp. 12 053–12 062.
- [30] T. Kuai, S. Honari, I. Gilitschenski, and A. Levinstein, "Towards unsupervised blind face restoration using diffusion prior," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025, pp. 1839–1849.
- [31] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6228–6237.
- [32] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [33] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [34] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [35] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *European Conference on Computer Vision (ECCV)*, 2016, pp. 694–711.
- [36] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [37] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4401–4410.
- [38] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-esrgan: Training real-world blind super-resolution with pure synthetic data," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2021, pp. 1905–1914.
- [39] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 586–595.
- [40] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [41] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *International Conference on Learning Representations (ICLR)*, 2018.



Fuchen Li Fuchen Li is currently pursuing the Second M.S. degree in Electrical and Computer Engineering with the Herbert Wertheim College of Engineering, University of Florida, Gainesville, FL, USA. Prior to his graduate studies, he worked as an image algorithm engineer in the imaging industry, focusing on AI-ISP, HDR imaging, and auto white balance for embedded camera systems. His research interests include low-level vision, computational photography, vision-language models, and edge intelligence.



Xinyang Wang Xinyang Wang received his B.E. degree in 2025 from the Department of Electronic Engineering at Shantou University, China. He is currently pursuing the M.E. degree in Electrical and Computer Engineering at the University of Florida. His research interests include machine learning, computer vision, and embedded systems.



Yahui Zhang Yahui Zhang is currently pursuing the Ph.D. degree at the University of Southampton, U.K. Her research interests include AI-generated images and visual culture studies.



Yuhan Chen Yuhan Chen received his master's degree in 2024 from the College of Mechanical Engineering at Chongqing University of Technology. He is currently pursuing the Ph.D. degree in the College of Mechanical and Vehicle Engineering at Chongqing University, China. His research interests include deep learning, low-level vision, and Gaussian splatting.



Jiahong Guo Jiahong Guo is currently pursuing the B.S. degree in Microbiology and Cell Science at the University of Florida, United States. His research interests in artificial intelligence in medical imaging, computer vision, deep learning, and dental image analysis. His current work focuses on intelligent medical imaging systems and the integration of artificial intelligence with dental and biomedical research.



Zhuohan Qin Zhuohan Qin is currently pursuing the M.S. degree in applied statistics with the School of Mathematics and Statistics, Qingdao University, Qingdao, China. Her research interests include computer vision, video understanding, and procedural action assessment.



Wenbo Ma Wenbo Ma is currently a Middleware Development Engineer with Intel Asia-Pacific Research & Development Ltd., Shanghai, China, where he focuses on cross-ISA kernel development, low-level hardware optimization, and AI high-performance computing.